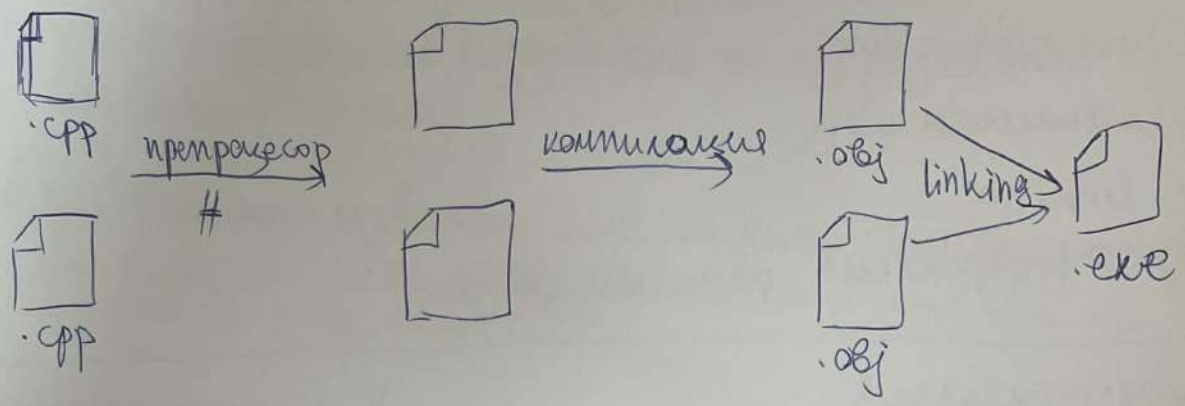
Тема 5

• разделна компилация



! компилират се само .cpp файловете, които са били променени след последната компилация

• синтаксис: [.h - само декларации (#pragma once !)
.cpp - имплементации]

1) препроцесор - текстообработка

#include "A.txt" - замества се със съдържанието на A.txt

<u>макроси</u> - заместване на текст с друг в целия код

#define st size_t

st size=10;

↳ size_t

inline f() - кодът във функцията се изпълнява на мястото на извикването ѝ

- не използва стековата рамка
- добре е да се ползва за "кратки" функции

2) синтактичен анализ

3) семантичен анализ

4) мендинни оптимизации

if(10>3) → няма нужда да се прави проверката

5) assembly код

6) машинен код

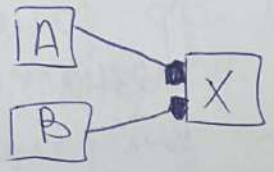
7) linking

8) CPU-dependent оптимизации

• композиция

X контролира жизнения цикъл на A и B.

```
class X {
  A a;
  B b;
};
```



A() B() X() → ~X() ~B() ~A()

• агрегация

A и B живеят извън X; X не контролира жизненият им цикъл

```
class X {
  A* a;
  B& b;
};
```

X() → ~X()

```
class Y {
  A a;
  B* b;
  C c;
};
```

Y() { b= new B(); } → Y е композиция (управлява B, заделяйки и трийки неговата памет)

~Y() { delete b; }

A() C() B() Y() → ~B() ~Y() ~C() ~A()

...пиращ конструктор

→ приема обект от същия клас и прави текущия негово копие

→ компилаторът го създава, ако не го разпишем (работи с тетерду)

→ при влагане извиква копиращите конструктори на всички член данни в началото си (т.е. се изпълнява след тях)

• оператор =

→ същото като к.к., но текущият обект вече съществува

→ компилаторът го създава

→ чисти текущите данни и копира новите

```
A obj1;
A obj2 = obj1;
```

→ извиква к.к. A(const A& other), а не op= (понеже obj2 тепърва се създава)

→ в op= винаги правим проверката!

```
A& op=(const A& other){
    if (this != &other)
    {
      //logic
    }
}
```

• (Named) Return value optimisation - (N)RVO

→ оптимизация, която спестява ~~ненужното~~ правене на

```
A create A() { return A(); }
A obj = createA();
```

↳ директно създава obj с default конструктора на A вместо да създаде обект в create A(), който после да копира

```
A createA() {
    A obj;
    :
    return obj;
}
```

→ NRVO - обектът не е създаден в return statement-а

• Пример с клас Event

.h

```
class Event {
    char name[20];
    Date date;
    Time start;
    Time end;

    void setName(const char*);
    void validateTimes();

public:
    Event();
    Event(const char*, const Date&, const Time&, const Time&);
    const char* getName() const;
    const Date& getDate() const;
    const Time& getStart() const;
    const Time& getEnd() const;
```